Exception Handling

Exception Handling is a programming mechanism used to detect and handle runtime errors so that the program does not crash and continues to run normally

Where error may occurs

There are 2 stages where error may happen in a program

- During compilation -> Syntax Error
- During execution -> Exceptions

Syntax Error ¶

- Something in the program is not written according to the program grammar.
- Error is raised by the interpreter/compiler
- You can solve it by rectifying the program

In [1]:
```
# example of syntax error

print 'hello world'
```

```
  Cell In[1], line 3
    print 'hello world'
    ^
SyntaxError: Missing parentheses in call to 'print'. Did you mean print(...)?
```

Other examples of syntax error

- Leaving symbols like colon,brackets
- Misspelling a keyword
- Incorrect indentation
- empty if/else/loops/class/functions

In [1]:
```
# Example of syntax error
a = 5
if a==3
print('hello')
```

```
  Cell In[1], line 2
    if a==3
           ^
SyntaxError: expected ':'
```

In [2]:
```
# Example of syntax error

a = 5
iff a==3:
print('hello')
```

```
  Cell In[2], line 2
    iff a==3:
        ^
SyntaxError: invalid syntax
```

In [3]:
```
# Example of syntax error

a = 5
if a==3:
print('hello')
```

```
  Cell In[3], line 3
    print('hello')
    ^
IndentationError: expected an indented block after 'if' statement on line 2
```

In [4]:
```
# IndexError
# The IndexError is thrown when trying to access an item at an invalid index.
L = [1,2,3]
L[100]
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[4], line 4
      1 # IndexError
      2 # The IndexError is thrown when trying to access an item at an invalid index.
      3 L = [1,2,3]
----> 4 L[100]

IndexError: list index out of range
```

In [5]:
```
# ModuleNotFoundError
# The ModuleNotFoundError is thrown when a module could not be found.
import mathi
math.floor(5.3)
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Cell In[5], line 4
      1 # ModuleNotFoundError
      2 # The ModuleNotFoundError is thrown when a module could not be found.
      3 # yaha math k badle mathi likh diya error show hoga kyuki iss naam se koi modu
le exist nahi karta
----> 4 import mathi
      5 math.floor(5.3)

ModuleNotFoundError: No module named 'mathi'
```

In [6]:
```
# KeyError
# The KeyError is thrown when a key is not found
# aap print karna chahte hoo d ka age jo ki iss dict me hai hee nahi -> key error
d = {'name':'nitish'}
d['age']
```

```
---------------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
Cell In[6], line 5
      1 # KeyError
      2 # The KeyError is thrown when a key is not found
      3 # aap print karna chahte hoo d ka age jo ki iss dict me hai hee nahi -> key er
ror
      4 d = {'name':'nitish'}
----> 5 d['age']

KeyError: 'age'
```

In [7]:
```
# TypeError
# The TypeError is thrown when an operation or function is applied
# to an object of an inappropriate type.
# Basically, you are mistyping something —
# the program is expecting one type of input, but you are providing a different input.

1 + 'a'
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[7], line 4
      1 # TypeError
      2 # The TypeError is thrown when an operation or function is applied to an objec
t of an inappropriate type.
      3 # basicaly aap miss type kar rahai ho jo axpect kar raha hai aap koi aur input
daal rahai hoo
----> 4 1 + 'a'

TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

In [8]:
```
# ValueError
# The ValueError is thrown when a function's argument is of an inappropriate type.
# For example, it is expecting an integer as input,
# but you provided a character instead.so the error will bw valueerror
int('a')
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[8], line 5
      1 # ValueError
      2 # The ValueError is thrown when a function's argument is of an inappropriate t
ype.
      3 # jaise ye integer koi input expect kar raha hai input hona chaiye int but apn
e character de diya
      4 # so the error will bw valueerror
----> 5 int('a')

ValueError: invalid literal for int() with base 10: 'a'
```

In [28]:
```
# NameError
# The NameError is thrown when an object could not be found.
# You tried to use a variable that you never created in your program.

print(k)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[28], line 5
      1 # NameError
      2 # The NameError is thrown when an object could not be found.
      3 # You tried to use a variable that you never created in your program.
----> 5 print(k)

NameError: name 'k' is not defined
```

In [27]:
```
# AttributeError
# Suppose you have a list and you want to run the upper() function on that list.
# You cannot apply the upper() function directly to a list because upper()
# is a function that works on strings.

L = [1,2,3]
L.upper()
# Stacktrace
```

```
---------------------------------------------------------------------------
AttributeError                            Traceback (most recent call last)
Cell In[27], line 7
      1 # AttributeError
      2 # Suppose you have a list and you want to run the upper() function on that lis
t.
      3 # You cannot apply the upper() function directly to a list because upper()
      4 # is a function that works on strings.
      6 L = [1,2,3]
----> 7 L.upper()

AttributeError: 'list' object has no attribute 'upper'
```

Exceptions

If things go wrong during the execution of the program(runtime). It generally happens when something unforeseen has happened.

- Exceptions are raised by python runtime
- You have to takle is on the fly

Examples

- Memory overflow
- Divide by 0 -> logical error
- Database error

In [11]:
```
# Why is it important to handle exceptions
# how to handle exceptions
# -> Try except block
```

In [13]:
```
# let's create a file
with open('sample.txt','w') as f:
    f.write('hello world')
```

In [14]:
```
# try catch demo
try:
    with open('sample1.txt','r') as f:
        print(f.read())
except:
    print('sorry file not found')
```

```
sorry file not found
```

In [16]:
```
# catching specific exception
try:
    m=5
    f = open('sample1.txt','r')
    print(f.read())
    print(m)
    print(5/2)
    L = [1,2,3]
    L[100]
except FileNotFoundError:
    print('file not found')
except NameError:
    print('variable not defined')
except ZeroDivisionError:
    print("can't divide by 0")
except Exception as e:
    print(e)
```

```
file not found
```

In [17]:
```
# else
try:
    f = open('sample1.txt','r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())
```

```
file nai mili
```

In [18]:
```
# finally
# else
try:
    f = open('sample1.txt','r')
except FileNotFoundError:
    print('file nai mili')
except Exception:
    print('kuch to lafda hai')
else:
    print(f.read())
finally:
    print('ye to print hoga hi')
```

```
file nai mili
ye to print hoga hi
```

In [19]:
```
# raise Exception
# In Python programming, exceptions are raised when errors occur at runtime.
# We can also manually raise exceptions using the raise keyword.

# We can optionally pass values to the exception to clarify why that exception
# was raised
```

In [20]:
```
raise ZeroDivisionError('aise hi try kar raha hu')
# Java
# try -> try
# except -> catch
# raise -> throw
```

```
---------------------------------------------------------------------------
ZeroDivisionError                         Traceback (most recent call last)
Cell In[20], line 1
----> 1 raise ZeroDivisionError('aise hi try kar raha hu')

ZeroDivisionError: aise hi try kar raha hu
```

In [22]:
```
class Bank:
    def __init__(self,balance):
        self.balance = balance
    def withdraw(self,amount):
        if amount < 0:
            raise Exception('amount cannot be -ve')
        if self.balance < amount:
            raise Exception('you have no money')
        self.balance = self.balance - amount
obj = Bank(10000)
try:
    obj.withdraw(15000)
except Exception as e:
    print(e)
else:
    print(obj.balance)
```

```
you have no money
```

In [25]:
```
def __init__(self,message):
    print(message)

class Bank:

    def __init__(self,balance):
        self.balance = balance
    def withdraw(self,amount):
        if amount < 0:
            raise MyException('amount cannot be -ve')
        if self.balance < amount:
            raise MyException('paise nai hai tere paas')
        self.balance = self.balance - amount
obj = Bank(10000)
try:
    obj.withdraw(5000)
except MyException as e:
    pass
else:
    print(obj.balance)
```

```
5000
```

In [24]:
```
# creating custom exceptions
# exception hierarchy in python
```

In [26]:

```python
# simple example

class SecurityError(Exception):

    def __init__(self,message):
        print(message)
    def logout(self):
        print('logout')

class Google:

    def __init__(self,name,email,password,device):
        self.name = name
        self.email = email
        self.password = password
        self.device = device
    def login(self,email,password,device):
        if device != self.device:
            raise SecurityError('bhai teri to lag gayi')
        if email == self.email and password == self.password:
            print('welcome')
        else:
            print('login error')

obj = Google('nitish','nitish@gmail.com','1234','android')
try:
    obj.login('nitish@gmail.com','1234','windows')
except SecurityError as e:
    e.logout()
else:
    print(obj.name)
finally:
    print('database connection closed')
```

```
bhai teri to lag gayi
logout
database connection closed
```

In [ ]: